

Lesson 2

Homework #1

Write a python program that prints the exact output provided below by using only one print statement:

Output:

Hello "World"!

What a beautiful day!

This is a great \time to write some Python code.

I'm excited to work on homework 2 as well.

Homework #2

Using the **sep** or/and **end** keywords, write a python program for each of the following statements:

1. Using **sep**, generate the following output:
Expected Output: Monday-Tuesday-Wednesday
2. Modify the previous statement to use a newline as the separator character instead of the hyphens (-).
3. Generate the following output by using one print statement for each of the inputs provided:
Input 1: "Hello"
Input 2: "World!"
Output: "Hello World!"

Homework #3

Write a python program that generates the following output:

Output:

```
  *
 ***
*****
*****
*****
*****
*****
***
  *
```

Rules:

1. Only **one print() call** is allowed.
2. No loops, no '*' * N multiplication.
3. You **must use sep and end creatively** to achieve the formatting.
4. Use **\n** inside the arguments to handle newlines.

Challenge #1

- Using the same rules in Homework #3, write a python program that generates the following output:

```
*  
***  
*****  
*****  
*****  
*****  
*****  
*****  
*****  
*****  
*****+*****  
*****++*****  
*****+*****  
*****+*****  
*****++++*****  
*****  
*****  
*****  
*****  
*****  
*****  
*****  
*****  
*****
```

Challenge #2

Write a python program that prints a perfectly formatted table given any input in the following format:

Expected Input:

```
c:name,age,score
e1:Mateo,15,65
e2:Lucas,34,49
e3:Penelope,27,85
```

The output must be a table like the one below:

Expected Output:

```
+-----+-----+-----+
| name  | age  | score |
+-----+-----+-----+
| Mateo | 15   | 65    |
| Lucas | 34   | 49    |
| Penelope | 27  | 85    |
+-----+-----+-----+
```

Challenge #2

- About the inputs:
 - The “c” represents the columns.
 - The “e#” represents each entry.
- Rules:
 1. The program must accept a command line argument indicating the number of entries to provide in the input.
 2. It must be able to accept any number of columns and rows(entries).
 3. For this challenge, feel free to use conditional statements, loops. Do not use any module that creates the table for you.

Challenge #2 – Usage Example

(ihl) > python challenge_2.py 3

Input:

Enter the column line (e.g. c:name,age,score):

c:name,age,score,word

Enter entry 1 (e1:...):

e1:Jorge,15,23,klk

Enter entry 2 (e2:...):

e2:Luis,34,49,delomio

Enter entry 3 (e3:...):

e3:Penelope,27,85,damelu!

Output:

```
+-----+-----+-----+-----+
|  name  | age | score |  word |
+-----+-----+-----+-----+
| Jorge  | 15 | 23 | klk   |
| Luis   | 34 | 49 | delomio |
| Penelope | 27 | 85 | damelu! |
+-----+-----+-----+-----+
```